

SOURCE CODE: -

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
/*
```

```
    Quick Sort Performance Evaluation
```

```
    -----
```

```
    Compares:
```

1. Quick Sort using the last element as pivot
2. Quick Sort using median-of-three sampling

```
    Median-of-three sampling selects three elements and  
    uses their median as the pivot.
```

```
*/
```

```
#define DATA_SIZE 1000000
```

```
// Swap two integers
```

```
void swap(int *a, int *b)
```

```
{
```

```
    int temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```

// Partition function
int partition(int arr[], int low, int high, int pivotIndex)
{
    int pivot = arr[pivotIndex];

    // Move selected pivot to the end
    swap(&arr[pivotIndex], &arr[high]);

    int i = low - 1;

    for (int j = low; j < high; j++)
    {
        if (arr[j] <= pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }

    swap(&arr[i + 1], &arr[high]);

    return i + 1;
}

// Standard Quick Sort using last element as pivot

```

```
void quickSortNormal(int arr[], int low, int high)
{
    if (low < high)
    {
        int pivotIndex = high;

        int pi = partition(arr, low, high, pivotIndex);

        quickSortNormal(arr, low, pi - 1);
        quickSortNormal(arr, pi + 1, high);
    }
}
```

```
// Return the median of three values
int medianOfThree(int arr[], int a, int b, int c)
{
    if ((arr[a] <= arr[b] && arr[b] <= arr[c]) ||
        (arr[c] <= arr[b] && arr[b] <= arr[a]))
        return b;

    if ((arr[b] <= arr[a] && arr[a] <= arr[c]) ||
        (arr[c] <= arr[a] && arr[a] <= arr[b]))
        return a;

    return c;
}
```

```
// Quick Sort using median-of-three sampling
void quickSortSampling(int arr[], int low, int high)
{
    if (low < high)
    {
        int mid = low + (high - low) / 2;

        // Select pivot using three samples:
        // first, middle and last elements
        int pivotIndex = medianOfThree(arr, low, mid, high);

        int pi = partition(arr, low, high, pivotIndex);

        quickSortSampling(arr, low, pi - 1);
        quickSortSampling(arr, pi + 1, high);
    }
}
```

```
// Generate random dataset
void generateData(int arr[], int n)
{
    for (int i = 0; i < n; i++)
    {
        arr[i] = rand();
    }
}
```

```
}
```

```
// Copy array
```

```
void copyArray(int source[], int destination[], int n)
```

```
{
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        destination[i] = source[i];
```

```
    }
```

```
}
```

```
// Check whether array is sorted
```

```
int isSorted(int arr[], int n)
```

```
{
```

```
    for (int i = 1; i < n; i++)
```

```
    {
```

```
        if (arr[i - 1] > arr[i])
```

```
            return 0;
```

```
    }
```

```
    return 1;
```

```
}
```

```
int main()
```

```
{
```

```
    int *original;
```

```
int *normal;

int *sampling;


clock_t start, end;


double normalTime;
double samplingTime;


printf("Quick Sort Performance Evaluation\n");
printf("=====\n");


printf("Dataset size: %d elements\n\n", DATA_SIZE);


// Allocate memory
original = (int *)malloc(DATA_SIZE * sizeof(int));
normal = (int *)malloc(DATA_SIZE * sizeof(int));
sampling = (int *)malloc(DATA_SIZE * sizeof(int));


if (original == NULL || normal == NULL || sampling == NULL)
{
    printf("Memory allocation failed.\n");
    return 1;
}


// Generate the same dataset for both algorithms
srand(42);
```

```
generateData(original, DATA_SIZE);

// -----
// Normal Quick Sort
// -----

copyArray(original, normal, DATA_SIZE);

start = clock();

quickSortNormal(normal, 0, DATA_SIZE - 1);

end = clock();

normalTime = (double)(end - start) / CLOCKS_PER_SEC;

printf("Normal Quick Sort completed.\n");
printf("Sorted correctly: %s\n",
       isSorted(normal, DATA_SIZE) ? "Yes" : "No");

printf("Execution Time: %.6f seconds\n\n", normalTime);

// -----
// Sampling Quick Sort
// -----
```

```

copyArray(original, sampling, DATA_SIZE);

start = clock();

quickSortSampling(sampling, 0, DATA_SIZE - 1);

end = clock();

samplingTime = (double)(end - start) / CLOCKS_PER_SEC;

printf("Sampling Quick Sort completed.\n");
printf("Sorted correctly: %s\n",
       isSorted(sampling, DATA_SIZE) ? "Yes" : "No");

printf("Execution Time: %.6f seconds\n\n", samplingTime);

// -----
// Comparison
// -----

printf("Performance Comparison\n");
printf("-----\n");

printf("Normal Quick Sort : %.6f seconds\n", normalTime);
printf("Sampling Quick Sort : %.6f seconds\n", samplingTime);

```



```
if (samplingTime < normalTime)
{
    printf("\nSampling-based Quick Sort was faster in this run.\n");
}
else if (samplingTime > normalTime)
{
    printf("\nNormal Quick Sort was faster in this run.\n");
}
else
{
    printf("\nBoth methods had approximately the same execution time.\n");
}

// Free memory
free(original);
free(normal);
free(sampling);

return 0;
}
```